Git

Controle de Versão Profissional no Dia a Dia

O que é, como funciona, comandos essenciais e boas práticas

#GIT #CONTROLEDEVERSÃO #DEVTOOLS

Made with GAMMA

O que é o Git?

Sistema de Controle de Versão

Git é um sistema de controle de versão. Ele registra todas as alterações feitas no código ao longo do tempo, permitindo voltar a versões anteriores, trabalhar em equipe sem conflito e manter um histórico completo do projeto.

📸 Salva "fotos" do código

Salva "fotos" do código em momentos específicos

👥 Trabalho em equipe

Permite que várias pessoas trabalhem no mesmo projeto

⏪ Voltar no tempo

Possibilita voltar a qualquer versão anterior

Made with GAMMA

Conceitos Fundamentais

📁 Repositório

Pasta do projeto controlada pelo Git

📸 Commit

Foto salva do código naquele momento

🌿 Branch

Ramificação para desenvolver sem afetar o principal

🔀 Merge

Unir duas branches em uma só

Made with GAMMA

Primeiros Passos — Configurando o Git

```
# Configurar nome e email (feito uma única vez)
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

# Verificar configurações
git config --list

# Iniciar um repositório novo
git init

# Clonar um repositório existente
git clone https://github.com/usuario/repositorio.git
```

💡 **Dica:** Sempre configure seu nome e email antes de começar — eles aparecem em cada commit.

Made with GAMMA

Os 3 Estados do Git

Working Staging Repositório

Pense assim: você **edita** → **prepara** → **salva**.

Made with GAMMA

Comandos Essenciais do Dia a Dia

Comando	Para que serve
`git status`	Ver o que foi modificado
`git add .`	Preparar todos os arquivos para commit
`git add arquivo.js`	Preparar um arquivo específico
`git commit -m "mensagem"`	Salvar as alterações com descrição
`git log`	Ver histórico de commits
`git diff`	Ver exatamente o que mudou
`git push origin branch`	Enviar commits para o repositório remoto
`git pull origin branch`	Baixar atualizações do repositório remoto

Made with GAMMA

Branches — Desenvolvendo Sem Afetar o Principal

```
# Criar uma nova branch
git branch feature-login

# Criar e já entrar na branch
git checkout -b feature-login

# Ver todas as branches
git branch

# Trocar de branch
git checkout nome-da-branch

# Deletar branch (após merge)
git branch -d feature-login
```

⚠️ Regra de Ouro

Nunca desenvolva direto na main — sempre crie uma branch para cada funcionalidade.

Made with GAMMA

Merge e Resolução de Conflitos

🔀 Fazer Merge

```
# Voltar para a main
git checkout main

# Unir a branch na main
git merge feature-login

# Enviar para o remoto
git push origin main
```

⚡ Conflito

Conflito acontece quando duas pessoas alteram o mesmo trecho de código. O Git marca o conflito assim:

```
<<<<<<< HEAD
seu código aqui
=======
código do colega
>>>>>>> feature-login
```

Escolha qual versão manter, salve e faça um novo commit.

Made with GAMMA

Como Escrever Boas Mensagens de Commit

❌ Exemplos Ruins

```
git commit -m "ajuste"
```

```
git commit -m "correção"
```

```
git commit -m "wip"
```

```
git commit -m "alterações"
```

✅ Exemplos Bons (Conventional Commits)

```
git commit -m "feat: adiciona login com JWT"
```

```
git commit -m "fix: corrige validação de email"
```

```
git commit -m "refactor: reorganiza estrutura de pastas"
```

```
git commit -m "docs: atualiza README"
```

💡 **Dica:** Use sempre: `tipo: descrição curta no imperativo`

Made with GAMMA

Git — O que Aprendemos

✅ `git init`

Inicia repositório

✅ `git add .`

Prepara arquivos

✅ `git commit -m ""`

Salva alterações

✅ `git push`

Envia para remoto

✅ `git pull`

Baixa atualizações

✅ `git branch`

Cria ramificação

✅ `git checkout`

Troca de branch

✅ `git merge`

Une branches

➡️ **Próximo:** GitHub — repositório remoto, Pull Requests e fluxo em equipe

Made with GAMMA